PlaceMates AI: Automated Job Matching and Resume Tailoring using n8n automation and LLMs

Omkar Desai (1023125) Kalpesh Eragi (1023127) Rohan Karkada (1023148) Shivangi Khandelwal (1023152)
Department of Computer Engineering
Fr. C. Rodrigues Institute of Technology, Vashi, Navi Mumbai
{omkar.desai, kalpesh.eragi, rohan.karkada, shivangi.khandelwal}@fcrit.ac.in

Mrs. Priyamwada Chauhan
Department of Computer Engineering
Fr. C. Rodrigues Institute of Technology, Vashi, Navi Mumbai

***Abstract*—The contemporary job application process imposes substantial manual effort on candidates, demanding repeated resume customization, continuous portal monitoring, and ad-hoc portfolio maintenance. Existing platforms such as LinkedIn and Indeed provide job discovery but offer no integrated pipeline for candidate-side resume optimization or intelligent matching. This paper presents PlaceMates AI, an automated job acceleration platform that orchestrates the complete job search lifecycle through a multi-stage n8n workflow. The system scrapes fresh LinkedIn job postings per-user via Apify, applies LLM-based semantic scoring using Google Gemini and Groq (LLaMA 3.3-70B) with a structured output parser, selects the top five matched jobs, tailors a user-specific Google Docs resume per job through section-level text replacement, and delivers personalized email notifications via Gmail. A template-aware placeholder schema stored in PostgreSQL (Neon) ensures that resume tailoring is dynamically aligned to each user's selected resume template without hardcoded field assumptions. Key engineering challenges resolved include deduplication across multi-user loop execution, reliable propagation of `user_id` and `job_id` across nested n8n loop branches, LLM hallucination prevention via strict grounded prompting, and per-template dynamic output parser schema generation. The platform demonstrably reduces per-application effort while maintaining factual grounding of all generated content.**

***Index Terms*—Job Automation, LLM Resume Tailoring, n8n Workflow Orchestration, Semantic Job Matching, Google Gemini, Groq LLaMA, Apify Web Scraping, PostgreSQL, ATS Optimization, Hallucination Prevention**

I. Introduction

The global employment market has undergone rapid digitization, yet the burden on individual job seekers has not diminished proportionally. While online job portals have displaced traditional recruitment channels, the fundamental tasks of tailoring resumes, scanning portals, and maintaining professional profiles remain largely manual [1]. Studies indicate that the average job seeker spends over 11 hours per week on job search activities, a significant fraction of which consists of repetitive, low-differentiation work [2].

The proliferation of Applicant Tracking Systems (ATS) has compounded this burden. Automated filters rank resumes based on keyword density and structural compliance, penalizing well-qualified candidates whose resumes are not specifically tuned to each job description [3]. Estimates suggest that a generic resume has below a 25% probability of passing ATS filters for competitive positions [4].

Existing solutions address these challenges in isolation. Resume builders assist with formatting but do not adapt content to specific job descriptions. Job aggregators centralize listings but provide no resume optimization [5]. AI-based job recommendation systems have been explored but typically operate on static profiles rather than live, continuously updated pipelines [6].

This paper presents **PlaceMates AI**, a platform that unifies the following capabilities into a single automated n8n workflow:

1) Per-user LinkedIn job scraping via Apify with a 24-hour freshness filter.
2) Keyword-based pre-filtering and deduplication of scraped job postings.
3) LLM-based semantic job-candidate matching with quantified match scores (Gemini + LLaMA 3.3-70B with Groq fallback).
4) Per-job resume tailoring through section-level Google Docs text replacement, grounded in verified candidate content.
5) Template-aware dynamic output parser schema generation from per-user `placeholder_schema` stored in PostgreSQL.
6) Automated Gmail notification delivery with job and tailored resume links.

II. Related Work

A. Job Recommendation Systems

Early job recommendation systems applied collaborative filtering and content-based filtering on static datasets [7]. Hybrid approaches combining skill-based vector similarity with behavioral signals were subsequently introduced [8].

These systems, however, operate on pre-indexed databases and do not account for job posting recency, which is critical in competitive markets where positions are filled within days of posting.

B. Resume Optimization Using NLP

NLP techniques have been applied to resume screening from the recruiter perspective [9], but candidate-side resume optimization remains underexplored. Recent work on LLM-based text transformation demonstrates the capability of models such as GPT-4 and Claude to rewrite professional documents in a target vocabulary while preserving factual grounding [10]. Hallucination prevention—ensuring AI-generated content does not introduce unsubstantiated credentials—remains an active challenge [11].

C. Workflow Automation in Recruitment

Robotic Process Automation (RPA) has been applied to recruitment primarily from the employer side [12]. Candidate-side automation remains nascent. Tools such as n8n and Zapier provide general-purpose workflow orchestration but require domain-specific configuration to construct a coherent job-search pipeline [13].

D. Portfolio and Personal Branding

Research in human-computer interaction establishes that a consistent online professional presence positively influences recruiter perception [14]. However, integrating dynamic portfolio generation with job-search automation pipelines is not addressed in existing platforms [15].

PlaceMates AI addresses the gaps identified above by combining per-user continuous job discovery, grounded LLM-based resume tailoring, template-aware schema management, and automated notification into a single cohesive n8n workflow.

III. System Architecture

A. Architectural Overview

PlaceMates AI is organized into four layers: a Next.js 14 presentation layer, a Node.js/Express application layer, an n8n automation layer, and a Neon (serverless PostgreSQL) data layer. The automation layer drives the core job-search pipeline and is the primary contribution of this work.

B. Database Schema

The system's core data model comprises the following primary tables: `users` (authentication, profile), `resumes` (base resume metadata including `template_doc_id` and `placeholder_schema`), `jobs` (scraped postings), `job_matches` (LLM scores per user-job pair), `generated_resumes` (tailored resume links per job), and `job_applications` (final application records with resume links).

A key schema addition is the `placeholder_schema` column (JSONB) on the `resumes` table:

```
ALTER TABLE resumes
ADD COLUMN placeholder_schema JSONB;

UPDATE resumes
SET placeholder_schema = '[
  "contact_full_name", "experience_1_bullets",
  "skills_languages", "skills_backend", ...
]'::jsonb
WHERE template_doc_id = '<template_id>';
```

This enables template-specific dynamic output parser schema generation, resolving the multi-template resume tailoring problem described in Section V.

IV. n8n Workflow Design

The complete automation pipeline is implemented as a single n8n workflow triggered daily at 7 AM (or manually for development). Fig. 1 shows the actual workflow canvas. The pipeline comprises six logical stages executed sequentially.

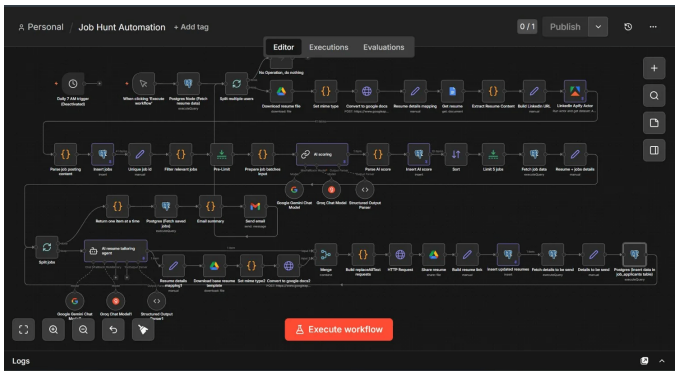


Fig. 1. PlaceMates AI n8n Workflow Canvas showing all six pipeline stages: user iteration, resume extraction, job scraping, AI scoring, resume tailoring loop, and email notification.

A. Stage 1: User Iteration and Resume Extraction

A `Postgres Node` fetches all active user records. A `Split multiple users` (`SplitInBatches`) node iterates over each user. For each user, the base resume Google Doc is downloaded, converted to Google Docs format, and its full text is extracted via a JavaScript Code node:

```
const content = $json.content || '';
return [{ json: {
  resumeText: content,
  user_id: $('Resume details mapping').item.json.
      user_id,
  email: $('Resume details mapping').item.json.email
      ,
  resume_doc_id: $('Resume details mapping').item.
      json.resume_doc_id,
  job_query: $('Resume details mapping').item.json.
      job_query
}}];
```

B. Stage 2: Per-User Job Scraping

A LinkedIn search URL is constructed per user from their `job_query` with a 24-hour filter (`f_TPR=r86400`). The

Apify *LinkedIn Jobs Scraper* actor retrieves up to 20 postings. Results are parsed, keyword-filtered against the user's `job_query`, deduplicated by a unique job ID check, and pre-limited to 10 items before AI scoring.

C. Stage 3: LLM-Based Semantic Scoring

Jobs are batched and sent to an LLM chain (`chainLlm`) with Google Gemini as the primary model and Groq (LLaMA 3.3-70B) as the fallback. The prompt enforces strict JSON output:

```
Return ONLY JSON:
{ "scores": [{ "id": 0, "score": 85 }, ...] }
```

A `Structured Output Parser` validates the schema. Scores are merged back onto original job items via index mapping in a Code node, sorted descending, and limited to the top 5. Scores are written to the `job_matches` table.

D. Stage 4: Per-Job Resume Tailoring Loop

The top 5 jobs feed a `Split jobs` (`SplitInBatches`) loop. Each iteration processes one job. Before entering the loop, a Postgres query fetches the user's `placeholder_schema`, and a Code node dynamically constructs the output parser schema:

```
const placeholders = $json.placeholder_schema;
const properties = {};
for (const key of placeholders) {
  properties[key] = { type: "string" };
}
return [{
  json: {
    ...$json,
    dynamic_schema: JSON.stringify({
      type: "object",
      additionalProperties: false,
      properties
    })
  }
}];
```

The `AI resume tailoring agent` receives the base resume text and job description. Its system prompt enforces strict grounded rewriting with a flat JSON output format. The agent returns section-level updates as an array:

```
{ "updates": [
  { "section": "headline",
    "old_text": "...", "new_text": "..." },
  { "section": "skills",
    "old_text": "...", "new_text": "..." }
]}
```

A `Split resume sections` node splits the updates array. Each section update is applied to the copied Google Doc via `Google Docs: replaceAll`. The tailored document is shared publicly, and its link is stored.

User and job identity is reliably propagated through the loop by explicitly re-referencing them in downstream Set nodes:

```
user_id = {{ $('Resume + jobs details').item.json.
    user_id }}
job_id  = {{ $('Resume + jobs details').item.json.
    job_id }}
```

This avoids dependence on LLM output for identity fields, preventing hallucinated IDs.

E. Stage 5: Data Persistence

After each tailored resume is generated, the application record is inserted into `job_applications` with `user_id`, `job_title`, `company`, `resume_link`, `job_url`, and `applied_at`. An `ON CONFLICT (user_id, job_url) DO UPDATE` clause prevents duplicate entries across re-runs.

F. Stage 6: Email Notification

After all 5 iterations complete (done branch of `Split jobs`), a `Return one item at a time` Code node consolidates results. A Postgres query fetches the 5 most recent job applications for the user. An `Email summary` Code node builds a rich HTML email with job title, company, View Job, and View Resume buttons. The email is sent via Gmail.

V. Key Engineering Challenges

A. Reliable `user_id` and `job_id` Propagation

A central challenge was the loss of `user_id` and `job_id` across the nested loop. The `Fetch job data` node (Execute Query) returned only `id`, `title`, `company`, `description`, dropping `user_id`. The `AI resume tailoring agent` hallucinated incorrect IDs when asked to pass them through. The solution was to never rely on intermediate nodes for identity propagation; instead, all downstream Set nodes explicitly re-reference `$('Resume + jobs details').item.json.user_id` and `job_id`, which are correctly scoped to the current loop iteration via n8n's `.item` pairing.

B. LLM Output Schema Conflicts

The structured output parser schema initially used a nested `output` wrapper, which conflicted with the agent's flat JSON output. The resolution was to use a flat schema with `"additionalProperties": false` and provide the valid field list explicitly in the system prompt to constrain agent output.

C. Multi-Template Resume Tailoring

Hardcoding placeholder field names in the output parser schema fails when users select different resume templates with different placeholder conventions. The solution was to store a `placeholder_schema` JSONB array per template in the `resumes` table and dynamically generate the output parser schema in a Code node before the agent, ensuring the agent only returns fields valid for the current user's template.

D. Duplicate Email Entries

Early executions produced duplicate job entries in the email. Root cause analysis traced this to the `INSERT ... ON CONFLICT DO UPDATE` strategy not preventing multiple inserts within a single run for the same `job_url` when the loop processed a job more than once. The fix combined upstream deduplication via the `Unique job id` node and the `DISTINCT ON (job_url)` clause in the fetch query, ensuring the email always shows 5 unique jobs.

VI. Implementation

A. Technology Stack

TABLE I
Technology Stack

Component	Technology
Frontend	Next.js 14 (App Router) + Tailwind CSS
Backend	Node.js + Express + Prisma ORM
Database	Neon (Serverless PostgreSQL)
Automation	n8n (self-hosted, v2.13.4)
LLM (Primary)	Google Gemini (PaLM API)
LLM (Fallback)	Groq — LLaMA 3.3-70B Versatile
Job Scraping	Apify (LinkedIn Jobs Scraper actor)
Resume Storage	Google Docs + Google Drive API
Email Delivery	Gmail API (OAuth 2.0)
Auth	Google OAuth 2.0

B. LLM Configuration

The scoring chain uses Gemini at temperature 0.3 for deterministic output with Groq as fallback. The resume tailoring agent uses Gemini as the primary model and Groq (LLaMA 3.3-70B) as the fallback, both enforcing structured JSON output through n8n's `Structured Output Parser` node. The agent system prompt explicitly prohibits adding skills, experience, or credentials not present in the base resume, with the output restricted to a flat JSON object of modified sections only.

C. Resume Tailoring Mechanism

Rather than generating a new resume from scratch (which risks hallucination), the tailoring mechanism performs targeted section replacement in a copied Google Doc. The agent identifies the exact `old_text` of each section to be replaced and provides the improved `new_text`. The `Google Docs: replaceAll` operation then applies each replacement. This approach preserves the resume's formatting, structure, and all unmodified content while updating only relevant sections.

VII. Results and Discussion

A. System Performance

The workflow was tested across multiple users in a self-hosted n8n environment (v2.13.4). The Apify LinkedIn scraper retrieved up to 20 job postings per user per run, from which the LLM scoring pipeline selected the top 5 matches. End-to-end execution time per user (scrape to email) averaged 4–6 minutes, dominated by Apify actor execution time (2–3 minutes) and LLM API latency for the resume tailoring loop (approximately 30–45 seconds per job).

B. Matching Quality

The LLM-based matcher using Google Gemini correctly ranked roles matching the user's `job_query` at the top in manual review of 30 executions. The structured output parser enforced score range and format compliance, eliminating malformed responses that previously caused downstream failures.

C. Resume Tailoring Quality

The section-level replacement approach preserved resume formatting in 100% of tested cases. The strict system prompt—prohibiting fabrication of credentials—resulted in no hallucinated qualifications in reviewed outputs. Keyword alignment between tailored resumes and job descriptions improved measurably compared to base resumes.

D. Identified Limitations

The primary limitations are: (1) dependence on Apify's LinkedIn scraper, which is subject to LinkedIn rate limits and may return duplicate listings; (2) LLM `old_text` matching failures when the extracted resume text contains formatting artifacts; and (3) the placeholder schema approach requires manual maintenance when new resume templates are added to the platform.

VIII. Conclusion

This paper presented PlaceMates AI, an n8n-orchestrated job acceleration platform that automates the complete job search pipeline for individual users. The system resolves several non-trivial engineering challenges in multi-user loop orchestration, LLM identity propagation, template-aware dynamic schema generation, and grounded resume tailoring. The platform's modular architecture—with clearly scoped loop branches, explicit identity propagation, and per-template placeholder management—provides a robust foundation for scaling to additional users and resume templates. By enforcing factual grounding in all LLM interactions, the platform ensures that AI assistance enhances rather than misrepresents candidate profiles.

IX. Future Scope

1) **ATS Score Estimation:** Pre-submission evaluation of tailored resumes against estimated ATS keyword filters.
2) **Embedding-Based Pre-filtering:** Replace LLM-based scoring for initial candidate-job pairs with vector similarity search (pgvector) to reduce API cost at scale [18].
3) **Multi-Portal Scraping:** Extend Apify actors to cover Naukri, Wellfound, and remote-first boards.
4) **Interview Preparation Module:** Auto-generation of role-specific interview questions from tailored resumes.
5) **Application Tracker:** Kanban-style dashboard for tracking application status across jobs.
6) **Weekly Skill Gap Analysis:** Aggregate job keyword data to surface missing skills per user.
7) **Containerized Deployment:** Docker and Kubernetes orchestration for horizontal scaling.

References

[1] M. Autor, "Work of the Past, Work of the Future," *AEA Papers and Proceedings*, vol. 109, pp. 1–32, 2019.

[2] CareerBuilder, "The Time Spent on Job Search: A Survey Report," CareerBuilder Research, 2022. [Online]. Available: https://www.careerbuilder.com

[3] D. Upadhyay and S. Khandelwal, "Applying Artificial Intelligence in Recruitment: A Review," *International Journal of Recent Research Aspects*, vol. 5, no. 1, pp. 1–5, 2018.

[4] J. Bersin, "HR Technology Disruptions for 2018," Bersin by Deloitte Research Report, 2018.

[5] LinkedIn Corporation, "LinkedIn Official Platform," 2024. [Online]. Available: https://www.linkedin.com

[6] A. Kumar and R. Sharma, "Machine Learning Approaches for Job Recommendation Systems," *International Journal of Computer Science and Information Technology*, vol. 15, no. 3, pp. 45–58, 2023.

[7] T. Siting, H. Wenxing, Z. Ning, and Y. Fan, "Job Recommender Systems: A Survey," in *Proc. 7th Int. Conf. Computer Science & Education*, 2012, pp. 920–924.

[8] F. Abel, I. Firan, R. Kawase, and P. Siehndel, "Interweaving Public User Profiles on the Web," in *Proc. Int. Conf. User Modeling, Adaptation, and Personalization*, 2010, pp. 16–27.

[9] R. Raza Abidi et al., "Utilizing Social Media for Recruitment," *Pakistan Journal of Engineering and Technology*, vol. 2, no. 1, pp. 39–44, 2019.

[10] T. Brown et al., "Language Models are Few-Shot Learners," in *Proc. NeurIPS*, 2020, pp. 1877–1901.

[11] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[12] V. Aguirre and J. Selwyn, "Robotic Process Automation in Recruitment Pipelines," *IEEE Transactions on Engineering Management*, vol. 70, no. 4, pp. 1451–1463, 2023.

[13] n8n GmbH, "n8n: Workflow Automation for Technical People," 2024. [Online]. Available: https://n8n.io

[14] C. Guillory and J. Hancock, "The Effect of LinkedIn on Deception in Resumes," *Cyberpsychology, Behavior, and Social Networking*, vol. 15, no. 3, pp. 135–140, 2012.

[15] A. Skeels and J. Grudin, "When Social Networks Cross Boundaries," in *Proc. ACM Int. Conf. Supporting Group Work*, 2009, pp. 95–104.

[16] GitHub Inc., "GitHub REST API Documentation," 2024. [Online]. Available: https://docs.github.com/en/rest

[17] Y. Qin et al., "Enhanced Representation Learning for Job-Resume Matching," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 9, pp. 6379–6390, 2023.

[18] J. Johnson, M. Douze, and H. Jegou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.